

1. K-Nearest Neighbors (KNN) Implementation

```

import numpy as np
from collections import Counter

class KNNFromScratch:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X_train, y_train):
        self.X_train = np.array(X_train)
        self.y_train = np.array(y_train)

    def _euclidean_distance(self, x1, x2):
        # Calculate Euclidean distance: sqrt(sum((x1 - x2)^2))
        return np.sqrt(np.sum((x1 - x2) ** 2))

    def _predict_instance(self, x):
        # 1. Compute Euclidean distance from test sample to all training samples
        distances = [self._euclidean_distance(x, x_train) for x_train in self.X_train]

        # 2. Get indices of the K nearest neighbors
        k_indices = np.argsort(distances)[:self.k]

        # 3. Extract the labels of the K nearest neighbors
        k_nearest_labels = [self.y_train[i] for i in k_indices]

        # 4. Return the most common label (majority voting)
        most_common = Counter(k_nearest_labels).most_common(1)
        return most_common[0][0]

    def predict(self, X_test):
        return np.array([self._predict_instance(x) for x in np.array(X_test)])

# --- Dataset and Execution ---
if __name__ == "__main__":
    # Small Synthetic Dataset (2 Features, Class 0 or Class 1)
    X_train = np.array([[1.0, 2.0], [1.5, 1.8], [5.0, 8.0], [6.0, 9.0], [1.0, 0.6], [9.0, 11.0]])
    y_train = np.array([0, 0, 1, 1, 0, 1])

    X_test = np.array([[1.2, 1.9], [5.5, 8.5], [8.5, 10.0], [0.8, 1.5]])
    y_test = np.array([0, 1, 1, 0])

    # User Input for K
    try:
        user_k = int(input("Enter the value of K: "))
    except ValueError:
        user_k = 3
    print("Invalid input! Defaulting to K = 3.")

    # Model Training and Prediction
    knn = KNNFromScratch(k=user_k)
    knn.fit(X_train, y_train)
    predictions = knn.predict(X_test)

    # Calculate Accuracy
    accuracy = np.sum(predictions == y_test) / len(y_test) * 100

    # Display Results
    print(f"\n--- KNN Classification Results (K={user_k}) ---")
    for i, (sample, pred, actual) in enumerate(zip(X_test, predictions, y_test)):
        print(f"Test Sample {i+1} {sample} -> Predicted Class: {pred} (Actual: {actual})")
    print(f"\nAccuracy: {accuracy:.2f}%")

```

Enter the value of K: 3

```

--- KNN Classification Results (K=3) ---
Test Sample 1 [1.2 1.9] -> Predicted Class: 0 (Actual: 0)
Test Sample 2 [5.5 8.5] -> Predicted Class: 1 (Actual: 1)
Test Sample 3 [ 8.5 10. ] -> Predicted Class: 1 (Actual: 1)
Test Sample 4 [0.8 1.5] -> Predicted Class: 0 (Actual: 0)

```

Accuracy: 100.00%

2. Logistic Regression Implementation

```

import numpy as np

class LogisticRegressionFromScratch:
    def __init__(self, learning_rate=0.1, num_iterations=1000):
        self.lr = learning_rate
        self.epochs = num_iterations
        self.weights = None
        self.bias = None

    def sigmoid(self, z):
        # Sigmoid function:  $g(z) = 1 / (1 + e^{-z})$ 
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0.0

        # Gradient Descent Optimization
        for epoch in range(self.epochs):
            # Linear combination  $z = Xw + b$ 
            linear_model = np.dot(X, self.weights) + self.bias
            # Sigmoid activation (probabilities)
            y_predicted = self.sigmoid(linear_model)

            # Gradient computation
            dw = (1 / n_samples) * np.dot(X.T, (y_predicted - y))
            db = (1 / n_samples) * np.sum(y_predicted - y)

            # Update parameters
            self.weights -= self.lr * dw
            self.bias -= self.lr * db

    def predict_prob(self, X):
        linear_model = np.dot(X, self.weights) + self.bias
        return self.sigmoid(linear_model)

    def predict(self, X, threshold=0.5):
        y_predicted_cls = [1 if p >= threshold else 0 for p in self.predict_prob(X)]
        return np.array(y_predicted_cls)

# --- Dataset and Execution ---
if __name__ == "__main__":
    # Linearly separable 2D dataset
    X_train = np.array([[0.5, 1.5], [1.0, 1.0], [1.5, 0.5], [3.0, 3.5], [4.0, 3.0], [3.5, 4.0]])
    y_train = np.array([0, 0, 0, 1, 1, 1])

    X_test = np.array([[0.8, 1.2], [3.2, 3.8], [2.0, 2.0], [1.2, 0.8]])
    y_test = np.array([0, 1, 1, 0])

    # Train Model
    lr_model = LogisticRegressionFromScratch(learning_rate=0.1, num_iterations=1000)
    lr_model.fit(X_train, y_train)

    # Get Sigmoid Output (Probabilities) and Binary Predictions
    probabilities = lr_model.predict_prob(X_test)
    predictions = lr_model.predict(X_test)

    # Evaluate Accuracy
    accuracy = np.sum(predictions == y_test) / len(y_test) * 100

    # Display Results
    print("--- Logistic Regression Results ---")
    for i, (sample, prob, pred, actual) in enumerate(zip(X_test, probabilities, predictions, y_test)):
        print(f"Sample {i+1} {sample} | Sigmoid Output: {prob:.4f} | Predicted Class: {pred} (Actual: {actual})")
    print(f"\nModel Accuracy: {accuracy:.2f}%")

```

```

--- Logistic Regression Results ---
Sample 1 [0.8 1.2] | Sigmoid Output: 0.0584 | Predicted Class: 0 (Actual: 0)
Sample 2 [3.2 3.8] | Sigmoid Output: 0.9831 | Predicted Class: 1 (Actual: 1)
Sample 3 [2. 2.] | Sigmoid Output: 0.4874 | Predicted Class: 0 (Actual: 1)
Sample 4 [1.2 0.8] | Sigmoid Output: 0.0576 | Predicted Class: 0 (Actual: 0)

```

Model Accuracy: 75.00%

